EPSI Montpellier
Option SLAM – Solutions Logicielles et Applications Métiers – 1ère année

RAPPORT DE STAGE

Conception et développement d'un site vitrine et d'un espace d'administration pour une boulangerie–pâtisserie artisanale

Le Fournil des Corbières

Stage du 5 mai 2026 au 31 mai 2026

Présenté par
Mathis GRAUET

Entreprise d'accueil : Le Fournil des Corbières
13 avenue Wilson, 11200 Lézignan-Corbières – 04 34 36 83 89

Année scolaire 2025–2026

Remerciements

Je tiens tout d'abord à remercier l'ensemble de l'équipe du **Fournil des Corbières** pour son accueil et pour la confiance qu'elle m'a accordée durant ces quatre semaines de stage. Je remercie en particulier mon maître de stage, le gérant de l'établissement, pour sa disponibilité, ses explications sur le fonctionnement quotidien de la boulangerie et la liberté qu'il m'a laissée pour proposer des solutions techniques adaptées à son activité.

Je souhaite également remercier l'ensemble de l'équipe pédagogique de **l'EPSI Montpellier**, et plus particulièrement mon tuteur de stage, pour le suivi assuré pendant cette période et pour les enseignements dispensés en première année de BTS SIO qui m'ont permis d'aborder ce projet avec les bases nécessaires en développement web, en gestion de bases de données et en méthodologie de projet.

Enfin, je remercie toutes les personnes qui, de près ou de loin, ont contribué au bon déroulement de ce stage et à la réalisation de ce rapport.

Sommaire

Introduction

Mon parcours et le contexte de ce stage

Présentation de l'entreprise d'accueil

La mission confiée

Organisation du rapport

Partie 1 – De l'analyse du besoin à la maquette du site vitrine

Partie 2 – Mise en place de l'environnement de développement et de la base de données

Partie 3 – Développement de l'espace d'administration et dynamisation du site

Partie 4 – Mise en ligne, tests et limites de la solution

Conclusion

Annexes

Introduction

Mon parcours et le contexte de ce stage

Actuellement en première année de **BTS Services Informatiques aux Organisations (SIO), option SLAM** – Solutions Logicielles et Applications Métiers – au sein de **l'EPSI Montpellier**, j'ai effectué un stage de quatre semaines, du **5 mai au 31 mai 2026**, au sein de l'entreprise **Le Fournil des Corbières**, une boulangerie-pâtisserie artisanale implantée à Lézignan-Corbières, dans l'Aude.

Ce stage constitue une étape obligatoire de ma formation : il permet de découvrir le milieu professionnel, de mettre en pratique les compétences acquises durant les premiers mois de cours et de valider une partie des compétences attendues dans le référentiel du BTS SIO, en particulier celles liées à la conception et au développement d'applications. Contrairement à un contrat de travail classique, le cadre de ce stage repose sur une convention de stage pédagogique signée conjointement par l'école, l'entreprise d'accueil et moi-même, et il a été réalisé selon une organisation hybride : quelques journées de présence à la boulangerie pour échanger avec mon maître de stage et observer l'activité, complétées par un travail à distance pour la partie développement, conformément aux modalités prévues dans la convention.

Présentation de l'entreprise d'accueil

Le **Fournil des Corbières** est un commerce de proximité situé au 13 avenue Wilson à Lézignan-Corbières (11200), ouvert du mardi au samedi (6h30 / 19h30) et dimanche (6h30 / 13h30). L'établissement propose une gamme de pains au levain naturel, de viennoiseries pur beurre, de pâtisseries traditionnelles et de pièces décoratives en chocolat ou en pâte d'amande, appelées « trompes-l'œil », qui mettent en valeur le savoir-faire artisanal de la maison.

Comme beaucoup de structures artisanales de cette taille, le Fournil des Corbières ne dispose pas d'un service informatique interne. Toute la partie numérique de l'activité – vitrine en ligne, mise en valeur des produits, communication – était jusqu'alors inexistante ou limitée. C'est dans ce contexte que le gérant, qui fait également office de maître de stage, a souhaité faire appel à un stagiaire en informatique pour donner à son commerce une présence sur le web.

La mission confiée

La mission qui m'a été confiée consistait à **concevoir et développer un site internet vitrine pour la boulangerie, ainsi qu'un espace d'administration permettant au gérant de gérer lui-même le contenu de son site sans connaissance technique particulière.** L'objectif final était double : d'une part, offrir aux clients une vitrine numérique attrayante présentant les spécialités de la maison (pains, pains spéciaux, pâtisseries et trompes-l'œil) ; d'autre part, donner au gérant un outil autonome lui permettant d'ajouter, de modifier, de réorganiser ou de supprimer ses produits – textes, photos et vidéos – au fil des saisons et de ses créations, sans avoir à solliciter de nouveau un développeur.

Au sein de cette petite structure, je me suis positionné comme le **prestataire informatique** du commerce : mon rôle était celui d'un **développeur web front-end**, chargé de concevoir et de réaliser une interface web vitrine responsive, puis de l'adosser à une base de données et à un panneau de gestion sécurisé. Cette mission m'a permis de couvrir une grande partie de la chaîne de production d'un site web professionnel : analyse du besoin, conception graphique, intégration HTML/CSS, modélisation et mise en place d'une base de données, développement PHP côté serveur, sécurisation des accès, puis mise en ligne et tests.

Organisation du rapport

Ce rapport retrace l'ensemble de la démarche suivie au cours de ce stage. La première partie présente la manière dont j'ai analysé le besoin du gérant et conçu la première version du site vitrine, en HTML et CSS. La deuxième partie détaille la mise en place de l'environnement de développement et de la base de données qui constitue le cœur technique du projet. La troisième partie est consacrée au développement de l'espace d'administration et à la dynamisation du site, c'est-à-dire au passage d'un site « figé » à un site piloté par une base de données. La dernière partie revient sur la mise en ligne du projet, les tests effectués et les limites de la solution retenue, avant de conclure sur le bilan de ce stage et les compétences qu'il m'a permis de développer.

Partie 1 – De l'analyse du besoin à la maquette du site vitrine

Avant d'écrire la moindre ligne de code, la première étape de la mission a consisté à comprendre précisément ce que mon maître de stage attendait de son futur site internet, puis à traduire ce besoin en une maquette HTML/CSS cohérente avec l'identité de la boulangerie. Cette partie présente la démarche suivie pour arriver à cette première version du site, ainsi que les choix techniques et graphiques qui en découlent.

1.1. Comprendre l'activité et les attentes du gérant

Les premiers échanges avec mon maître de stage ont permis de cerner les besoins réels de l'entreprise. Le Fournil des Corbières ne souhaitait pas un site de vente en ligne complexe – aucun système de commande ou de paiement n'était demandé – mais bien un **site vitrine**, c'est-à-dire une carte de visite numérique présentant l'établissement, ses horaires, son adresse et surtout ses produits phares, dans un style visuel chaleureux et artisanal.

Cette analyse du besoin m'a permis de dégager plusieurs contraintes structurantes pour la suite du projet :

- présenter les spécialités de la maison (pains, viennoiseries, pâtisseries, trompes-l'œil) de manière visuelle et attractive ;
- mettre en avant l'histoire et l'ancrage local de la boulangerie, entre mer et montagnes, pour valoriser le caractère artisanal de la maison ;
- afficher clairement les informations pratiques : adresse, horaires d'ouverture, contact, réseaux sociaux ;
- proposer un site fonctionnant aussi bien sur ordinateur que sur smartphone, la majorité des clients consultant ce type d'information depuis leur téléphone ;
- permettre au gérant de faire évoluer le contenu du site (nouveaux produits, nouvelles photos) de façon autonome – ce dernier point n'a été développé que dans une seconde phase du projet, présentée en partie 3.

1.2. Choix techniques pour la maquette initiale

Pour cette première phase, j'ai opté pour une approche volontairement simple et standard : un site statique en **HTML5 et CSS3**, sans framework, développé sous **Visual Studio Code**. Ce choix se justifie par plusieurs raisons. D'abord, il s'agit des langages de base enseignés en première année de BTS SIO, ce qui me permettait de mobiliser directement les compétences acquises en cours. Ensuite, pour un site vitrine de cette taille, un framework JavaScript aurait représenté une complexité inutile, à la fois pour le développement et pour la maintenance future par un développeur tiers. Enfin, partir d'un code HTML/CSS « pur » facilitait l'intégration progressive de PHP dans les phases suivantes du projet.

Dès cette étape, j'ai appliqué une bonne pratique de développement consistant à **séparer la structure du contenu (HTML) de sa présentation (CSS)**, en plaçant l'ensemble des règles de mise en forme dans une feuille de style dédiée plutôt que dans des balises `<style>` intégrées à la page. Cette séparation, certes simple, a un intérêt concret : si le gérant souhaite plus tard modifier une couleur ou une police, il suffit de modifier un seul fichier, sans toucher au contenu des pages.

1.3. Conception graphique : du choix de l'ambiance à la maquette retenue

Pour la dimension graphique, j'ai proposé à mon maître de stage plusieurs orientations visuelles possibles pour habiller la même structure de contenu, afin qu'il puisse choisir celle qui correspondait le mieux à l'image qu'il souhaitait donner de son commerce. Quatre univers graphiques ont ainsi été esquissés – une version classique et chaleureuse, une version plus « chic », une version mettant l'accent sur les matières naturelles, et une version au style vintage – chacune reposant sur une palette de couleurs et une typographie différentes, mais sur la même architecture de page.

Après présentation de ces pistes, mon maître de stage a retenu la version **« Classique et Chaleureuse »**, jugée la plus rassurante pour une clientèle de quartier. Cette identité visuelle repose sur une palette de couleurs directement inspirée de l'univers de la boulangerie :

- `#5d4037` – un brun « pain bien cuit », utilisé comme couleur principale pour l'en-tête, le pied de page et les titres ;
- `#f3e5ab` – une couleur « blé / farine », utilisée comme couleur secondaire pour les fonds de section ;
- `#fdf5e6` – un ton crème, utilisé pour le fond général du site, afin de rappeler la chaleur du four et la mie de pain ;
- une police sans-serif sobre (« Segoe UI »), choisie pour sa bonne lisibilité sur tous les écrans.

Sur le plan de la mise en page, la structure retenue pour la page d'accueil reprend les codes classiques d'un site vitrine, présentés dans le tableau ci-dessous.

Section	Contenu
En-tête (header)	Image de fond immersive, nom de la boulangerie, slogan et bouton d'appel à l'action vers la section Contact.
Navigation	Menu fixe (« sticky ») qui reste visible lors du défilement, avec des liens vers les sections Pains, Pâtisseries, Trompes-l'œil et Notre Histoire.
Grille de produits	Présentation des spécialités sous forme de cartes (image, nom, courte description), organisées en grille responsive.
Notre Histoire	Bloc texte présentant l'identité et l'ancrage local de la boulangerie.
Pied de page (footer)	Coordonnées, horaires, lien vers les réseaux sociaux et, à terme, accès discret à l'espace d'administration.

Tableau 1 – Structure de la page d'accueil retenue pour la maquette « Classique et Chaleureuse ».

1.4. Une grille de produits responsive

La grille de produits constitue l'élément central de la page d'accueil, puisque c'est elle qui met en valeur le travail du boulanger-pâtissier. Pour qu'elle s'adapte automatiquement à la taille de l'écran – ordinateur, tablette ou smartphone – sans avoir à écrire de code spécifique pour chaque format, j'ai utilisé la propriété CSS `display: grid` associée à la fonction `repeat(auto-fit, minmax(280px, 1fr))`. Cette technique permet au navigateur de calculer lui-même le nombre de cartes à afficher par ligne en fonction de l'espace disponible, tout en garantissant une largeur minimale de 280 pixels par carte.

Chaque carte produit (« `.product-card` ») a été conçue comme un bloc autonome composé d'un cadre image de hauteur fixe et d'une zone de texte (nom du produit et courte description). Pour que l'image remplisse entièrement ce cadre sans être déformée, quelle que soit la proportion de la photo d'origine, j'ai utilisé la propriété CSS `object-fit: cover`, qui « recadre » intelligemment l'image pour qu'elle occupe tout l'espace disponible. Un léger effet de survol (`:hover`) fait également « remonter » la carte de quelques pixels, pour donner au site une impression de modernité et de réactivité.

1.5. Vers une gestion sémantique et accessible des images

La première version de la grille de produits affichait les photos comme de simples arrière-plans CSS, déclarés directement dans la feuille de style (`background-image: url(...)`). Si cette approche fonctionnait visuellement, elle présentait deux limites importantes que j'ai identifiées avec mon maître de stage : d'une part, elle obligeait à modifier le fichier CSS chaque fois que l'on souhaitait changer une photo, ce qui n'était pas envisageable pour un gérant non technicien ; d'autre part, une image « cachée » dans le CSS n'est pas reconnue comme un contenu par les moteurs de recherche ni par les technologies d'assistance pour les personnes malvoyantes.

J'ai donc fait évoluer le code pour utiliser la balise HTML `<img>`, accompagnée systématiquement d'un attribut `alt` décrivant le produit (par exemple « Le Pavé des Corbières »). Cette évolution, en apparence modeste, répond à trois objectifs :

- **autonomie** : il devient possible de changer une photo simplement en remplaçant un fichier image, sans toucher au code ;
- **accessibilité et référencement** : chaque image devient un contenu identifiable, avec une description textuelle ;
- **préparation de la suite du projet** : cette structure, où chaque carte associe une image, un nom et une description, est directement réutilisable lorsque ces informations proviendront ensuite d'une base de données (voir partie 3).

1.6. Une première vérification d'accessibilité : le contraste des couleurs

En relisant la première version du site, j'ai remarqué que les liens – par exemple le lien vers la page Instagram de la boulangerie, situé dans le pied de page – conservaient la couleur bleue par défaut des navigateurs (`#0000EE`), peu lisible sur le fond marron foncé (`#5d4037`) du footer. Ce contraste insuffisant pose un problème d'accessibilité au sens des normes **WCAG** (Web Content Accessibility Guidelines), qui recommandent un contraste minimal entre le texte et son arrière-plan pour garantir la lisibilité par tous les utilisateurs, y compris les personnes malvoyantes.

J'ai corrigé ce point en définissant pour ces liens une couleur crème (`#f3e5ab`) cohérente avec la palette du site, accompagnée d'un effet de survol qui souligne le lien et le fait passer en blanc. Ce détail illustre une démarche que j'ai essayé de conserver tout au long du stage : ne pas se contenter d'un rendu « qui fonctionne », mais vérifier que celui-ci respecte également les bonnes pratiques du développement web, même sur un projet de petite taille.

1.7. Bilan de cette première phase

À l'issue de cette première phase, le Fournil des Corbières disposait d'une maquette de site vitrine fonctionnelle, responsive et conforme à l'identité visuelle souhaitée, organisée en deux fichiers distincts (`acceuil.html` et `acceuil.css`). Ce socle, bien que purement statique – les produits affichés étaient encore « écrits en dur » dans le code HTML – constituait une base solide pour la suite du projet. Les parties suivantes présentent comment ce site a évolué pour devenir une véritable application web dynamique, pilotée par une base de données et administrable par le gérant lui-même.


Le Fournil des Corbières
L'authenticité du pain artisanal au cœur du terroir.
Nous trouver
ACCUEIL
NOS PAINS
PÂTISSERIES
NOTRE HISTOIRE
CONTACT
Nos Spécialités
Du levain naturel, des farines locales et beaucoup de passion.

Partie 2 – Mise en place de l'environnement de développement et de la base de données

Une fois la maquette statique validée, l'étape suivante consistait à transformer ce site « vitrine figée » en une véritable application capable de stocker des informations et de les restituer dynamiquement. Cette partie présente l'architecture technique retenue, les choix d'outils effectués et la manière dont j'ai modélisé puis mis en place la base de données du projet, ainsi que les premières fonctions d'authentification qui en découlent.

2.1. Vers une architecture trois couches

Pour qu'un site puisse afficher du contenu qui change – par exemple, la liste des produits proposés par la boulangerie – sans que cela nécessite de réécrire le code HTML à chaque fois, il faut introduire deux nouveaux éléments : un langage de programmation côté serveur, capable de générer dynamiquement les pages, et une base de données, capable de stocker durablement les informations. Ce principe correspond à **l'architecture trois couches**, un modèle classique en développement web qui distingue :

- **la couche présentation** (le client) : le code HTML et CSS, c'est-à-dire ce que voit et avec quoi interagit le visiteur du site ;
- **la couche logique** (le serveur) : le code **PHP**, qui reçoit les requêtes du navigateur, exécute les traitements nécessaires et construit les pages à la volée ;
- **la couche données** : la **base de données**, qui conserve durablement les informations (produits, comptes administrateurs, etc.).

Pour que ces trois couches puissent communiquer, il faut qu'un « moteur » capable d'exécuter du PHP soit en fonctionnement sur la machine de développement. Cette notion, abstraite au premier abord, s'est concrètement traduite par une première difficulté technique rencontrée en tout début de cette phase : en tentant de me connecter à une base de données depuis l'extension MySQL de VS Code, j'ai obtenu une erreur `ECONNREFUSED`, signifiant qu'aucun serveur de base de données n'était démarré sur le port attendu (3306). Cette erreur, bien que frustrante au premier abord, m'a permis de bien comprendre ce que recouvrait concrètement la notion d'architecture trois couches, et de réfléchir au choix du système de gestion de base de données le plus adapté au projet.

2.2. Le choix de SQLite plutôt qu'un serveur MySQL

Le choix « par défaut » pour un environnement PHP est généralement **MySQL**, associé à un environnement « tout-en-un » comme WampServer ou XAMPP, qui fait tourner en arrière-plan un véritable serveur de base de données. Cette solution est robuste, mais elle introduit une dépendance forte : sans ce serveur démarré, rien ne fonctionne, et le projet devient plus difficile à déplacer d'une machine à l'autre.

Pour un projet de la taille de celui du Fournil des Corbières – quelques dizaines de produits tout au plus, un seul compte administrateur, et un seul utilisateur simultané – j'ai fait le choix de **SQLite**, un système de gestion de base de données « embarqué ». Contrairement à MySQL, SQLite ne nécessite **aucun serveur** : toute la base de données est contenue dans un unique fichier (`boulangerie.db`), que PHP peut lire et écrire directement, via l'extension standard `PDO` (PHP Data Objects).

Ce choix présente plusieurs avantages pour ce contexte précis : légèreté (un simple fichier à sauvegarder ou à déplacer), portabilité (le projet fonctionne de la même façon sur n'importe quel poste disposant de PHP), et simplicité de mise en œuvre pour un développement en autonomie. Il s'agit d'un compromis assumé : SQLite serait moins adapté à un site à fort trafic avec de nombreux utilisateurs simultanés, mais il correspond parfaitement aux besoins d'un site vitrine pour une boulangerie de quartier.

2.3. Mise en place de l'environnement local : VS Code, PHP et SQLite

La mise en place concrète de cet environnement de développement s'est faite entièrement dans Visual Studio Code, en utilisant :

- l'extension **PHP Server**, qui permet de lancer un serveur PHP local en un clic de souris (équivalent de la commande `php -S localhost:8000`) ;
- l'extension **SQLite Viewer**, pour visualiser le contenu du fichier `boulangerie.db` directement dans l'éditeur, sans logiciel supplémentaire ;
- l'outil externe **DB Browser for SQLite**, utilisé en complément pour créer ou modifier des tables avec une interface graphique, jouant un rôle équivalent à celui de PhpMyAdmin pour MySQL.

La mise en route de cet environnement n'a pas été immédiate. La première tentative de connexion à SQLite depuis VS Code s'est traduite par une erreur `could not find driver`, indiquant que PHP ne disposait pas des extensions nécessaires pour dialoguer avec SQLite. La résolution de ce problème a nécessité d'ouvrir le fichier de configuration `php.ini` et d'y activer deux lignes, désactivées par défaut, en supprimant le point-virgule qui les mettait en commentaire :

```
;extension=pdo_sqlite        →   extension=pdo_sqlite
;extension=sqlite3           →   extension=sqlite3
```

Après modification de ce fichier, un redémarrage du serveur PHP a permis de faire disparaître l'erreur. Cette manipulation, bien que technique, m'a permis de mieux comprendre le rôle du fichier `php.ini` dans la configuration d'un environnement PHP, ainsi que l'importance de toujours redémarrer un serveur après modification de sa configuration – un réflexe que j'ai retrouvé à plusieurs reprises pendant le stage.

2.4. Organisation de l'arborescence du projet

Avant de créer la base de données, j'ai défini une arborescence de dossiers claire pour le projet, en suivant le principe consistant à séparer les ressources selon leur nature. Cette organisation, présentée dans le tableau ci-dessous, a été conservée – avec quelques évolutions présentées en partie 3 – jusqu'à la fin du projet.

Dossier / fichier	Rôle
acceuil.php	Page d'accueil du site, point d'entrée pour les visiteurs.
css/acceuil.css	Feuille de style générale du site.
data/boulangerie.db	Fichier de base de données SQLite (tables admin et produits).
fonction/	Ensemble des scripts PHP de gestion (connexion, authentification, panneau d'administration, gestion des produits).
fonction/img/, fonction/video/	Photos et vidéos des produits ajoutées via le panneau d'administration.

Tableau 2 – Arborescence retenue pour le projet « Le Fournil des Corbières ».

Cette organisation répond à deux objectifs : la **clarté**, chaque type de ressource ayant son propre dossier (images, vidéos, scripts, styles, données), et la **sécurité**, le fait d'isoler le fichier de base de données dans un dossier `data/` dédié facilitant, à terme, la mise en place de restrictions d'accès direct à ce dossier depuis un navigateur.

2.5. Modélisation des données : les tables admin et produits

La base de données du projet repose sur deux tables principales, dont la structure a été affinée au fil du développement. La table `admin` stocke les informations nécessaires à la connexion du gérant à son espace de gestion ; la table `produits` stocke l'ensemble des articles présentés sur le site.

Table / champ	Type	Rôle
admin		
id	INTEGER (clé primaire)	Identifiant unique du compte.
identifiant	VARCHAR(50)	Identifiant de connexion (ex. « patron »).
mot_de_passe	VARCHAR(255)	Mot de passe, stocké sous forme hachée.
produits		
id	INTEGER (clé primaire)	Identifiant unique du produit.
nom	VARCHAR(100)	Nom du produit (ex. « Le Pavé des Corbières »).
description	TEXT	Courte description affichée sous le nom du produit.
categorie	VARCHAR(50)	Catégorie du produit : pain, patisseries, trompe_oeil ou speciaux.
image_url	VARCHAR(255)	Chemin vers le fichier image ou vidéo associé.

Table / champ	Type	Rôle
type_media	VARCHAR(20)	Type de média : « image » ou « video » (ajouté en cours de projet).
ordre	INTEGER	Position d'affichage du produit dans sa catégorie (ajouté en cours de projet).

Tableau 3 – Modèle de données retenu pour la base « boulangerie.db » (le schéma détaillé figure en annexe 2).

Cette table `produits` n'a pas été figée dès le départ : les champs `type_media` et `ordre` ont été ajoutés en cours de projet, au fur et à mesure que de nouveaux besoins ont émergé (gestion des vidéos, puis réorganisation manuelle des produits, présentées en partie 3). Pour ajouter une colonne à une table déjà existante sans perdre les données qu'elle contient, j'ai utilisé l'instruction SQL `ALTER TABLE produits ADD COLUMN ...`. J'ai par ailleurs pris soin d'écrire ces scripts de mise à jour de manière **idempotente**, c'est-à-dire qu'ils vérifient d'abord, via l'instruction `PRAGMA table_info(produits)`, si la colonne existe déjà avant de tenter de la créer. Cette précaution évite qu'une exécution répétée du script ne provoque une erreur « colonne déjà existante », qui aurait fait planter le site à chaque chargement de page si le code de migration avait été placé au mauvais endroit.

2.6. Sécurisation des accès : authentification et hachage du mot de passe

Le gérant n'ayant besoin que d'un seul compte d'administration, j'ai opté pour un **accès restreint** plutôt que pour un système d'inscription complet : il n'existe aucune page permettant de créer un nouveau compte, et le compte unique du gérant est créé directement dans la base de données via un script PHP dédié, exécuté une seule fois.

Point essentiel du référentiel BTS SIO sur la sécurité des accès, le mot de passe n'est **jamais stocké en clair** dans la base de données. Il est transformé grâce à la fonction PHP `password_hash()`, qui produit une empreinte non réversible du mot de passe. Lors de la connexion, la fonction `password_verify()` compare le mot de passe saisi avec cette empreinte, sans jamais avoir besoin de « déchiffrer » quoi que ce soit. Ainsi, même en cas d'accès non autorisé au fichier `boulangerie.db`, le mot de passe réel du gérant ne serait pas directement lisible.

Le mécanisme d'authentification repose sur trois fichiers, dont le rôle et l'enchaînement sont résumés ci-dessous :

Fichier	Rôle
fonction/login.php	Affiche le formulaire de connexion (identifiant / mot de passe).
fonction/auth.php	Récupère les informations saisies, interroge la table admin via une requête préparée, vérifie le mot de passe avec password_verify(), puis ouvre une session PHP si la vérification réussit.

Fichier	Rôle
fonction/admin_panel.php	Page protégée : vérifie en début de script qu'une session administrateur est active (sinon, redirige vers login.php), puis affiche l'espace de gestion.

Tableau 4 – Enchaînement des fichiers assurant l'authentification de l'administrateur.

L'ensemble des requêtes interrogeant la base de données utilise des **requêtes préparées** (via PDO, avec des paramètres représentés par des points d'interrogation ?), plutôt que de construire directement une chaîne SQL à partir des données saisies par l'utilisateur. Cette pratique, systématique dans le projet, permet de se protéger des injections SQL, une faille de sécurité classique consistant à faire passer du code SQL malveillant via un formulaire.

2.7. Bilan de cette deuxième phase

À l'issue de cette phase, le projet disposait d'une base de données opérationnelle, d'un environnement de développement fonctionnel et d'un premier système d'authentification sécurisé, donnant accès à une page d'administration encore minimale (« Bienvenue Patron ! »). Cette page, accessible depuis un lien discret placé dans le pied de page du site, constitue le point de départ de l'espace de gestion détaillé dans la partie suivante, où le panneau d'administration et l'affichage du site ont progressivement été reliés à la base de données pour devenir réellement dynamiques.


Accès Fournil
Identifiant
Mot de passe
Entrer

Partie 3 - Developpement de l'espace d'administration et dynamisation du site

Cette troisième partie correspond au cœur du développement réalisé pendant le stage : il s'agissait de transformer le site statique conçu en partie 1, adossé à la base de données mise en place en partie 2, en une **application web dynamique**, où le contenu affiché aux visiteurs provient entièrement de la base de données et peut être géré par le gérant via un espace d'administration sécurisé. Cette partie présente successivement la gestion des produits (ajout, modification, suppression), la dynamisation de la page d'accueil, la gestion des médias - photos et vidéos - et enfin les fonctionnalités de confort ajoutées à l'espace d'administration.

3.1. Concevoir l'espace d'administration : ajouter, modifier, supprimer

La première étape a consisté à donner au gérant la possibilité d'**ajouter de nouveaux produits** à son catalogue. J'ai créé une page `fonction/ajouter_produit.php`, accessible uniquement depuis le panneau d'administration, présentant un formulaire avec les champs suivants : nom du produit, description, catégorie (choisie dans une liste déroulante : Pains, Pâtisseries, Trompes-l'œil, Pains spéciaux) et image associée. À la validation de ce formulaire, le script PHP exécute une requête `INSERT INTO produits (...)` pour enregistrer le nouveau produit dans la base de données.

La fonctionnalité de **modification de produit** (page `modifier_produit.php`) repose sur le même principe, mais utilise une requête `UPDATE produits SET ... WHERE id = ?`. Le panneau d'administration affiche une liste déroulante permettant au gérant de choisir le produit à modifier ; le formulaire se pré-remplit alors avec les informations actuelles de ce produit. Lors de la mise au point de cette fonctionnalité, j'ai veillé à harmoniser le nom de la variable utilisée pour identifier le produit (« id_produit ») entre le formulaire HTML et le script PHP qui le traite : une incohérence de nommage entre les deux fichiers est une source d'erreur fréquente et particulièrement difficile à repérer, puisqu'elle ne provoque généralement aucun message d'erreur explicite - le formulaire semble « ne rien faire ».

Pour la **suppression d'un produit** (page `supprimer_produit.php`), j'ai porté une attention particulière à l'expérience utilisateur et à la sécurité de l'action, en ajoutant une **double confirmation** : un premier clic sur le bouton « Supprimer » fait apparaître, via un court script JavaScript, deux nouveaux boutons « Confirmer » et « Annuler », le bouton initial étant masqué. Ce n'est qu'en cliquant sur « Confirmer » que la requête `DELETE FROM produits WHERE id = ?` est réellement exécutée. Cette précaution, simple à mettre en œuvre, évite qu'un clic accidentel ne supprime définitivement un produit - et la photo associée.

Pour conserver une page d'administration lisible malgré la multiplication des produits, j'ai organisé la liste des produits existants sous forme d'un **accordéon**, à l'aide des balises HTML natives `<details>` et `<summary>`. Chaque catégorie devient ainsi un menu repliable affichant le nombre de produits qu'elle contient, sans nécessiter de bibliothèque JavaScript supplémentaire. Les

produits sont par ailleurs triés par ordre alphabétique grâce à la clause SQL `ORDER BY nom ASC`, ce qui facilite la recherche d'un produit donné.

3.2. Dynamiser la page d'accueil

Une fois le panneau d'administration capable d'enregistrer des produits, il restait à faire en sorte que la page d'accueil les affiche automatiquement. Cette étape a nécessité de renommer le fichier `acceuil.html` en `acceuil.php` - condition indispensable pour que le serveur interprète le code PHP qu'il contient - puis de remplacer chaque bloc de produits « écrit en dur » par une **boucle PHP** parcourant les résultats d'une requête `SELECT * FROM produits WHERE categorie = '...'`.

```
<div class="produits-grille">
  <?php
  $req = $db->query("SELECT * FROM produits WHERE categorie = 'pain'");
  while ($p = $req->fetch()) {
  ?>
    <div class="product-card">
      <div class="product-img-frame">
        <img src="fonction/<?= $p['image_url'] ?>"
             alt="<?= $p['nom'] ?>">
      </div>
      <div class="product-info">
        <h3><?= $p['nom'] ?></h3>
        <p><?= $p['description'] ?></p>
      </div>
    </div>
  <?php } ?>
</div>
```

Extrait - Boucle d'affichage dynamique des produits de la catégorie « pain » sur la page d'accueil.

Cette transformation a mis en évidence un type d'erreur fréquent en PHP, et révélateur du fonctionnement du langage : si une balise `<?php ... ?>` reste ouverte par erreur, ou si le fichier ne porte pas l'extension `.php`, le serveur ne tente pas d'exécuter le code et affiche son contenu « tel quel », sous forme de texte brut, directement dans la page - ce qui s'est notamment produit lors d'un oubli de renommage du fichier. Comprendre ce comportement m'a permis, par la suite, de diagnostiquer plus rapidement ce type d'incident : lorsqu'un code PHP s'affiche « en texte » sur la page plutôt que de s'exécuter, la cause est presque toujours liée à l'extension du fichier ou à une balise d'ouverture/fermeture PHP mal placée.

La page d'accueil finale comporte ainsi **quatre sections dynamiques**, correspondant chacune à une catégorie de produits (Pains, Pains spéciaux, Pâtisseries, Trompes-l'œil), chacune étant alimentée par sa propre requête SQL filtrée sur la colonne `categorie`. La catégorie « Pains spéciaux » a été ajoutée en cours de projet pour répondre à une demande de mon maître de stage de pouvoir distinguer les pains « du quotidien » des créations plus originales (pains aux graines, pains spéciaux saisonniers, etc.).

3.3. Gérer l'envoi et le rangement des photos de produits

La fonctionnalité la plus exigeante techniquement de cette phase a été la **gestion de l'envoi de fichiers (upload)** depuis le formulaire d'ajout de produit. Contrairement à la simple saisie de texte, l'envoi d'une image nécessite : de configurer le formulaire HTML avec l'attribut `enctype="multipart/form-data"` (sans lequel le fichier n'est tout simplement pas transmis au serveur), de récupérer le fichier côté PHP via le tableau superglobal `$_FILES`, puis de le déplacer du dossier temporaire du serveur vers son emplacement définitif avec la fonction `move_uploaded_file()`.

J'ai par ailleurs ajouté plusieurs vérifications avant d'accepter un fichier :

- vérification de l'extension du fichier, en la comparant à une liste d'extensions autorisées (jpg, jpeg, png, gif, webp pour les images, mp4, mov, webm pour les vidéos) ;
- nettoyage du nom de fichier d'origine, à l'aide de la fonction `preg_replace()`, afin de supprimer les espaces, accents et caractères spéciaux susceptibles de provoquer des erreurs (un fichier envoyé depuis un smartphone porte par exemple souvent un nom de type « Photo Instagram (1).jpg ») ;
- ajout d'un horodatage (`time()`) au nom du fichier final, afin d'éviter qu'un nouveau fichier ne porte exactement le même nom qu'un fichier déjà existant et ne l'écrase involontairement.

Une difficulté récurrente : les chemins de fichiers relatifs

La gestion des **chemins de fichiers** a constitué la difficulté la plus récurrente de cette phase, et l'une des plus instructives. Un chemin relatif, comme `../img/`, est interprété par PHP **non pas par rapport à l'emplacement du fichier HTML affiché dans le navigateur, mais par rapport à l'emplacement du script PHP qui s'exécute**. Or, le script `ajouter_produit.php` se trouvant dans le dossier `fonction/`, tandis que la page `acceuil.php` se trouve à la racine du projet, un même chemin relatif ne désignait pas le même dossier selon le fichier qui l'utilisait - ce qui s'est traduit, à plusieurs reprises, par des images correctement enregistrées sur le serveur mais affichées « cassées » sur le site (icône d'image manquante).

La solution retenue a consisté à **centraliser le stockage des médias dans le dossier** `fonction/`, avec deux sous-dossiers `fonction/img/` et `fonction/video/`, et à utiliser systématiquement la constante PHP `__DIR__`, qui renvoie toujours le chemin absolu du dossier contenant le script en cours d'exécution, pour construire les chemins de destination des fichiers. Côté affichage, la page `acceuil.php` préfixe simplement le chemin enregistré en base par `fonction/`. Cette difficulté, bien que pénible à déboguer dans l'instant, m'a permis de comprendre une notion fondamentale du développement web côté serveur : la distinction entre le **chemin d'exécution** (où se trouve le script PHP) et le **chemin d'URL** (ce que voit le navigateur).

Enfin, lorsqu'un produit est supprimé ou que son média est remplacé lors d'une modification, le script appelle la fonction `unlink()` pour supprimer le fichier image ou vidéo devenu inutile du

serveur, après avoir vérifié son existence avec `file_exists()`. Cette « maintenance automatique » du dossier de médias évite l'accumulation de fichiers orphelins au fil du temps.

3.4. Permettre la gestion de vidéos pour les trompes-l'œil

Une fois la gestion des photos opérationnelle, mon maître de stage a exprimé le souhait de pouvoir ajouter de **courtes vidéos** pour certains produits, en particulier les trompes-l'œil, dont l'aspect spectaculaire - une pièce en chocolat ou en pâte d'amande imitant un objet du quotidien - se prête particulièrement bien à une mise en mouvement. Cette évolution a nécessité plusieurs adaptations :

- ajout de la colonne `type_media` à la table `produits` (voir partie 2.5), permettant de préciser si le média associé à un produit est une image ou une vidéo ;
- ajout, dans le formulaire d'administration, d'un menu déroulant « Type de média » (Image / Vidéo) ;
- dans la boucle d'affichage de la page d'accueil, ajout d'une condition PHP (`if ($p['type_media'] === 'video')`) permettant d'afficher soit une balise `<img>`, soit une balise `<video>` (avec les attributs `autoplay muted loop playsinline` pour une lecture automatique, silencieuse et en boucle, discrète et adaptée à une vitrine).

La première tentative d'envoi de vidéo s'est heurtée à une erreur « fichier non valide ». L'investigation a montré que cette erreur pouvait avoir plusieurs origines : une taille de fichier dépassant la limite par défaut de PHP (2 Mo), définie par les paramètres `upload_max_filesize` et `post_max_size` du fichier `php.ini`, ou une extension de fichier non reconnue par le script de vérification. Augmenter ces limites (à 50 Mo, par exemple) et élargir la liste des extensions autorisées aux formats vidéo courants (mp4, mov, webm) a permis de résoudre ce problème.

Sur le plan de la mise en page, l'ajout de l'élément `<video>` a momentanément perturbé l'alignement de la grille de produits : un lecteur vidéo ne possède pas, par défaut, les mêmes propriétés d'affichage qu'une image, ce qui « cassait » la mise en page en `flexbox` du CSS existant. J'ai résolu ce point en appliquant explicitement à `.product-img-frame img` et `.product-img-frame video` les mêmes propriétés (`width: 100%; height: 100%; object-fit: cover`), garantissant un comportement identique quel que soit le type de média affiché.

Pour réduire le poids des vidéos envoyées par mon maître de stage - souvent filmées directement avec un smartphone, en haute définition - je lui ai par ailleurs recommandé d'utiliser un outil de compression gratuit comme *HandBrake*, au format « MP4 Web-Optimized (H.264) », qui réduit considérablement la taille du fichier sans perte de qualité visible, pour garantir un chargement rapide du site même sur une connexion mobile.

3.5. Une fenêtre « zoom » (lightbox) pour les produits

Pour valoriser davantage les vidéos et photos de produits, en particulier celles des trompes-l'œil, j'ai ajouté une fonctionnalité de **fenêtre modale (lightbox)** : un clic sur n'importe quelle carte

produit ouvre une fenêtre agrandie, superposée au reste de la page, affichant le média en grand format avec son titre et sa description. Cette fenêtre peut être fermée en cliquant sur une croix, en cliquant à l'extérieur de la fenêtre, ou en appuyant sur la touche `Échap` du clavier.

Cette fonctionnalité a été développée en JavaScript, et j'ai pris le parti de l'extraire dans un fichier dédié `script.js`, séparé du code HTML de `acceuil.php`, dans la continuité de la logique de séparation des préoccupations déjà appliquée au CSS en partie 1.

Une difficulté inattendue est apparue lors des premiers tests sur les vidéos en plein écran : la vidéo, une fois agrandie, **redémarrait depuis le début et saccadait** légèrement avant de devenir fluide. L'analyse de ce problème a permis de comprendre que mon code recréait, à chaque clic, un nouvel élément `<video>`, obligeant le navigateur à recharger entièrement le fichier en haute définition. La solution consiste à **cloner** l'élément vidéo déjà présent sur la page - et donc déjà chargé en mémoire par le navigateur - grâce à la méthode JavaScript `cloneNode(true)`, puis à insérer ce clone, déjà prêt, dans la fenêtre agrandie. Ce détail, invisible pour l'utilisateur final lorsqu'il fonctionne correctement, illustre bien la différence entre un code « qui marche » et un code « agréable à utiliser », et m'a sensibilisé à l'importance des performances perçues, en particulier sur des contenus vidéo.

3.6. Réorganiser librement l'ordre des produits (glisser-déposer)

La dernière fonctionnalité ajoutée à l'espace d'administration répond à une demande très concrète de mon maître de stage : pouvoir **changer l'ordre d'affichage des produits** au sein d'une même catégorie - par exemple, mettre en avant la nouveauté de la semaine en la plaçant en premier - sans avoir à modifier chaque produit individuellement.

Pour répondre à ce besoin, j'ai mis en place un système de **glisser-déposer (drag & drop)** sur une nouvelle page d'administration (`voir_produits.php`), à l'aide de la bibliothèque JavaScript *SortableJS*, chargée depuis un CDN. Le fonctionnement repose sur trois éléments :

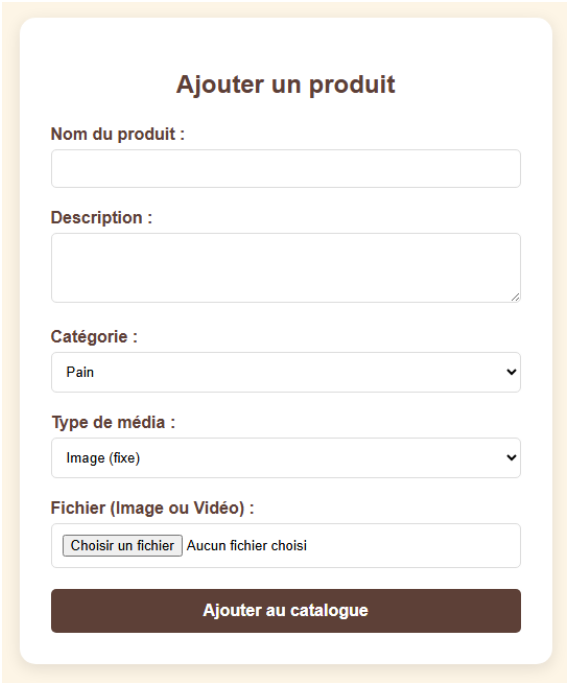
- la colonne `ordre` ajoutée à la table `produits` (voir partie 2.5), utilisée pour trier les produits affichés ;
- la bibliothèque SortableJS, appliquée à chaque grille de produits, qui rend les cartes « attrapables » (le curseur de la souris change pour symboliser cette action) et gère visuellement le réordonnancement par glisser-déposer ;
- un nouveau script `save_order.php`, appelé automatiquement en arrière-plan via la fonction JavaScript `fetch()` dès qu'un produit est déposé à sa nouvelle place, qui met à jour la colonne `ordre` de chaque produit concerné dans la base de données.

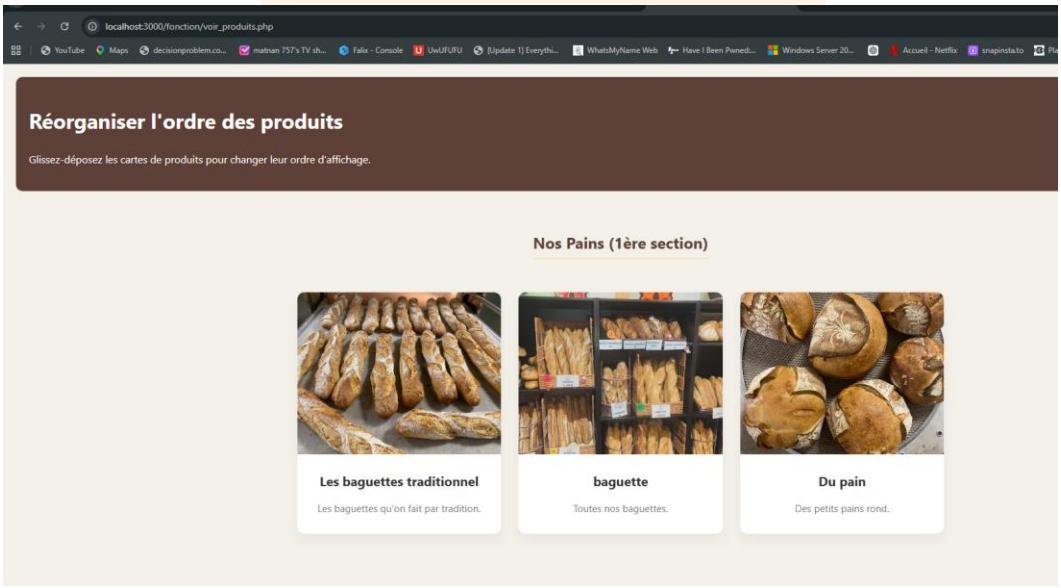
Cette fonctionnalité illustre la notion d'**AJAX** (envoi de données au serveur sans recharger la page) : le gérant voit le résultat de son action instantanément, tandis que l'enregistrement en base de données s'effectue « silencieusement » en arrière-plan. La page d'administration a également été enrichie d'une feuille de style dédiée (`admin.css`) et d'icônes issues de la

bibliothèque *Font Awesome*, afin de donner à cet espace, initialement très sommaire, une présentation plus professionnelle, cohérente avec l'identité visuelle du site public.

3.7. Bilan de cette troisième phase

Cette phase, la plus longue du stage, a permis de faire passer le projet d'un site « affichant des informations » à un véritable **outil de gestion de contenu sur mesure** : le gérant du Fournil des Corbières dispose désormais d'un espace d'administration sécurisé lui permettant d'ajouter, de modifier, de supprimer et de réorganiser ses produits - photos comme vidéos - sans intervention extérieure, le site public se mettant à jour automatiquement. La partie suivante présente comment ce projet, jusqu'alors développé et testé en local, a été préparé pour une mise en ligne, ainsi que les limites et perspectives d'évolution qui en découlent.





Partie 4 - Mise en ligne, tests et limites de la solution

Une fois les fonctionnalités de gestion de contenu opérationnelles, j'ai souhaité vérifier, avec mon maître de stage, dans quelle mesure le site pouvait être mis à disposition d'un véritable public sur Internet. Cette dernière partie présente les tests réalisés tout au long du projet, la démarche de mise en ligne entreprise, ainsi que les limites identifiées et les pistes d'évolution envisagées pour la solution.

4.1. Une démarche de test continue

Plutôt que de réserver une phase de tests à la toute fin du projet, j'ai adopté une démarche de **vérification continue**, chaque nouvelle fonctionnalité étant testée aussitôt après son développement, directement en environnement local via le serveur PHP intégré (`php -S localhost:8000`). Ce choix s'explique par la nature même du projet : à chaque étape, une nouvelle brique technique (base de données, authentification, upload de fichiers, gestion vidéo, glisser-déposer) venait s'ajouter aux précédentes, et une régression sur une fonctionnalité antérieure aurait pu facilement passer inaperçue si les tests avaient été reportés.

Concrètement, chaque test suivait le même protocole : se connecter à l'espace d'administration avec le compte du gérant, ajouter ou modifier un produit de test, vérifier son apparition immédiate sur la page d'accueil avec le bon visuel et la bonne catégorie, puis vérifier que le fichier média correspondant avait bien été enregistré au bon endroit sur le serveur. Cette dernière vérification - regarder directement dans l'explorateur de fichiers de VS Code - a été particulièrement utile pour diagnostiquer les problèmes de chemins de fichiers présentés en partie 3.3.

Sur le plan de la sécurité, j'ai également vérifié qu'il était impossible d'accéder directement aux pages de gestion (`admin_panel.php`, `ajouter_produit.php`, etc.) en tapant simplement leur adresse dans le navigateur sans être connecté : chaque script vérifie en début d'exécution la présence d'une session administrateur active, et redirige le cas échéant vers la page de connexion. Lors de ces tests, j'ai identifié un bug provenant d'une configuration de session incohérente entre les différents fichiers PHP : selon l'ordre dans lequel `session_start()` et `require_once 'db.php'` étaient appelés, la session pouvait être perdue d'une page à l'autre, déconnectant le gérant de manière intempestive. La correction a consisté à harmoniser, au tout début de chaque script protégé, l'ordre et la configuration de ces instructions, en y ajoutant `session_set_cookie_params(['path' => '/'])` pour garantir que la session reste valable sur l'ensemble du site, quel que soit le dossier depuis lequel un script est exécuté.

4.2. Préparer la mise en ligne : versionnement et hébergement

Pour préparer une éventuelle mise en ligne, j'ai mis en place un **dépôt Git** pour le projet, hébergé sur GitHub. Cette étape a soulevé une question de fond, que j'ai posée à mon maître de stage avant d'agir : un dépôt GitHub public rendrait visible, par n'importe qui, non seulement le code source du site, mais aussi le contenu du fichier `boulangerie.db`, c'est-à-dire la liste complète des

produits *et le mot de passe haché de l'administrateur*. Or, ce site n'étant pas le mien mais celui d'un client, sa mise en ligne sans précaution aurait posé un problème de confidentialité des données de l'entreprise.

J'ai donc créé un **dépôt GitHub privé** (visible uniquement par moi et, sur autorisation, par mon maître de stage), accompagné d'un fichier `.gitignore` excluant les fichiers système inutiles. Cette démarche m'a permis d'aborder, de manière concrète, une question que je n'avais jusque-là rencontrée que de manière théorique en cours : la différence entre le code source d'une application (qui peut, selon les cas, être partagé) et les données qu'elle manipule (qui appartiennent au client et doivent rester confidentielles).

Pour tester un déploiement réel, j'ai ensuite relié ce dépôt privé à la plateforme **Render**, un service d'hébergement proposant une offre gratuite avec déploiement automatique à chaque envoi de code (`git push`). L'exécution de PHP sur Render nécessitant une configuration plus poussée que sur un hébergeur PHP traditionnel, j'ai dû écrire un `Dockerfile`, c'est-à-dire un fichier de configuration décrivant un environnement d'exécution complet (ici, une image PHP avec serveur Apache et les extensions SQLite nécessaires), conteneurisé avec *Docker*. Si cette étape sortait du cadre strict de la mission initiale, elle m'a permis une première découverte, certes partielle, des notions de conteneurisation et de déploiement continu, de plus en plus présentes dans les environnements professionnels.

4.3. La limite principale identifiée : la persistance des données en ligne

La mise en ligne sur Render a permis de mettre en évidence la **limite technique la plus importante** de l'architecture choisie. Sur l'offre gratuite de cet hébergeur, le système de fichiers du site est *éphémère* et passe en lecture seule après le démarrage : à chaque redéploiement ou redémarrage du service, tous les fichiers sont restaurés dans l'état où ils se trouvent sur le dépôt Git. Or, le fichier `boulangerie.db` étant précisément le fichier que l'espace d'administration doit pouvoir modifier (ajout, suppression, réorganisation de produits), toute tentative d'écriture depuis le site en ligne échouait avec une erreur `attempt to write a readonly database`.

Cette limite n'est pas un défaut de mon code, mais une **contrainte structurelle** liée au choix initial de SQLite (un fichier local) combiné à une offre d'hébergement gratuite à système de fichiers éphémère. Deux solutions ont été étudiées pour y répondre :

- **migrer vers une base de données externe** (par exemple PostgreSQL, via un service comme Neon.tech), accessible depuis le code PHP via une variable d'environnement `DATABASE_URL`, ce qui aurait nécessité d'adapter le fichier `db.php` pour qu'il sache se connecter soit à SQLite en local (pour le développement), soit à PostgreSQL en ligne (pour la production) ;
- **conserver SQLite, mais sur un disque persistant**, c'est-à-dire un espace de stockage de l'hébergeur qui n'est pas réinitialisé lors des redémarrages.

Après discussion avec mon maître de stage sur le coût et la complexité de chaque option, la **seconde solution** a été retenue : l'ajout d'un disque persistant (option payante, mais d'un coût très modeste) a permis de conserver l'architecture SQLite, en modifiant simplement le fichier `db.php` pour qu'il pointe vers ce nouvel emplacement (`/data/boulangerie.db` plutôt que `data/boulangerie.db`), et en adaptant le `Dockerfile` pour que ce dossier dispose des bons droits d'écriture. Le script de création des tables, écrit de manière idempotente (voir partie 2.5), s'est révélé particulièrement utile à cette étape : au tout premier démarrage sur ce nouveau disque, il a recréé automatiquement la structure de la base de données, sans aucune intervention manuelle.

Une dernière difficulté est apparue à ce stade : Git ne suit pas les dossiers vides. Les dossiers `data/`, `fonction/img/` et `fonction/video/`, ne contenant à l'origine aucun fichier versé sur le dépôt (les photos et vidéos des produits étant volontairement exclues du code source), n'étaient tout simplement pas créés lors du déploiement, ce qui empêchait le serveur d'y écrire. La solution, classique mais peu intuitive au premier abord, a consisté à placer dans chacun de ces dossiers un fichier caché nommé `.gitkeep`, dont le seul rôle est de forcer Git à conserver le dossier, même vide.

4.4. Synthèse des principales difficultés rencontrées

Le tableau ci-dessous récapitule, de manière transversale, les principales difficultés techniques rencontrées au cours du stage et les solutions qui y ont été apportées. Au-delà de leur résolution individuelle, ces difficultés partagent un point commun : elles relèvent presque toutes de la **configuration de l'environnement** (PHP, serveur, extensions, hébergement) plutôt que de la logique métier du code lui-même - un constat qui, avec le recul, me paraît représentatif d'une grande partie du travail réel d'un développeur, au-delà de la seule écriture de code.

Difficulté rencontrée	Cause identifiée	Solution apportée
Erreur ECONNREFUSED à la connexion à la base de données	Aucun serveur de base de données démarré sur le poste	Choix de SQLite (sans serveur dédié)
Erreur could not find driver	Extensions pdo_sqlite / sqlite3 désactivées dans php.ini	Activation des extensions et redémarrage du serveur PHP
Code PHP affiché en texte brut sur la page	Fichier en .html au lieu de .php, ou balise PHP non fermée	Renommage en .php et vérification des balises d'ouverture/fermeture
Images cassées après upload	Chemin relatif interprété depuis le dossier du script (fonction/), pas depuis la racine	Centralisation des médias dans fonction/ et usage de __DIR__
Erreur fichier non valide pour les vidéos	Limite upload_max_filesize / extension non autorisée	Augmentation des limites dans php.ini et ajout des extensions vidéo
Vidéo saccadée en plein écran (lightbox)	Élément vidéo recréé à chaque ouverture, rechargement complet	Clonage de l'élément existant via cloneNode(true)

Difficulté rencontrée	Cause identifiée	Solution apportée
Déconnexions intempestives de l'administrateur	Configuration de session incohérente entre les fichiers PHP	Harmonisation de session_start() et session_set_cookie_params()
Écriture impossible sur la base de données en ligne	Système de fichiers éphémère de l'hébergement gratuit	Ajout d'un disque persistant et adaptation de db.php

Tableau 5 - Synthèse des principales difficultés rencontrées et des solutions apportées.

4.5. Limites de la solution et pistes d'évolution

Au-delà de la question de l'hébergement, traitée ci-dessus, plusieurs limites peuvent être identifiées dans la solution actuelle, ouvrant la voie à des évolutions futures du projet :

- **absence de sauvegarde automatique** : le fichier `boulangerie.db` constitue, à lui seul, l'ensemble du catalogue de produits. Une sauvegarde régulière de ce fichier (par exemple via une tâche planifiée copiant le fichier vers un autre emplacement) serait une amélioration simple et utile ;
- **compte administrateur unique** : si l'activité du Fournil des Corbières venait à évoluer (plusieurs personnes amenées à gérer le site), il faudrait réintroduire une gestion multi-comptes dans la table `admin`, avec des rôles éventuellement différents ;
- **absence de redimensionnement automatique des images** : les photos envoyées depuis un smartphone sont conservées dans leur résolution d'origine, ce qui peut ralentir le chargement du site. Un redimensionnement automatique à l'envoi (via la bibliothèque GD de PHP, par exemple) améliorerait les performances sans intervention du gérant ;
- **pas de tableau de bord statistique** : le gérant ne dispose d'aucune indication sur la fréquentation de son site. L'intégration d'un outil d'analyse d'audience respectueux de la vie privée (par exemple Plausible ou Matomo) pourrait constituer une évolution intéressante.

Ces limites n'ont pas été traitées dans le cadre de ce stage, par manque de temps, mais elles ont été consignées et évoquées avec mon maître de stage, qui dispose désormais d'un projet documenté (ce rapport, ainsi que l'historique des modifications conservé sur le dépôt Git) pouvant servir de base à un futur développeur pour faire évoluer le site.

Conclusion

Bilan des compétences techniques acquises

Ce stage de quatre semaines au sein du Fournil des Corbières m'a permis de mener, de bout en bout, un projet de développement web complet : de l'analyse du besoin d'un client jusqu'à une première mise en ligne, en passant par la conception graphique, la modélisation d'une base de données, le développement d'un espace d'administration sécurisé et la gestion de contenus multimédias. Sur le plan strictement technique, ce stage a été l'occasion de mettre en pratique, dans un cas réel, un ensemble de notions abordées en cours mais rarement réunies dans un même projet :

- **développement front-end** : HTML5 sémantique, CSS3 (mise en page en grille, flexbox, responsive design, accessibilité) et JavaScript (manipulation du DOM, événements, requêtes AJAX) ;
- **développement back-end** : PHP procédural, gestion de formulaires, gestion de fichiers (upload, suppression), sessions ;
- **bases de données** : modélisation d'un schéma relationnel simple, écriture de requêtes SQL (création de tables, insertion, mise à jour, suppression, filtrage), utilisation de PDO et des requêtes préparées, migrations de schéma idempotentes ;
- **sécurité** : hachage des mots de passe, contrôle d'accès par session, protection contre les injections SQL ;
- **outils et environnement** : Visual Studio Code et ses extensions, configuration de PHP (php.ini), versionnement avec Git et GitHub, première approche du déploiement via Render et Docker.

Au-delà de ces compétences techniques prises individuellement, c'est surtout leur articulation au sein d'un même projet qui m'a le plus appris. Comprendre, par exemple, qu'un simple changement de couleur de lien (partie 1.6) renvoie à une norme d'accessibilité, ou qu'une erreur de chemin de fichier (partie 3.3) renvoie à la différence entre architecture client et architecture serveur, m'a aidé à relier des notions théoriques, vues séparément en cours, à des situations concrètes.

Apport de ce stage pour ma formation en BTS SIO

Ce stage trouve un écho direct avec plusieurs blocs de compétences du référentiel du BTS SIO, option SLAM, et notamment ceux relatifs à la conception et au développement d'applications. La mission confiée m'a permis de mobiliser, dans un contexte professionnel réel et avec un client présent, des compétences d'analyse de besoin, de conception (choix d'une architecture, modélisation de données), de réalisation (développement front-end et back-end) et de mise en œuvre (tests, déploiement).

Ce stage m'a également permis de mesurer la différence entre un exercice scolaire, généralement cadré et aux consignes précises, et une mission professionnelle, où le besoin du client évolue au fil du projet : la demande initiale - un simple site vitrine en HTML/CSS - s'est progressivement enrichie, à la demande de mon maître de stage, jusqu'à intégrer un espace d'administration complet avec gestion de photos, de vidéos et de l'ordre d'affichage des produits. Cette évolution progressive du besoin, plutôt qu'un défaut de cadrage initial, m'a semblé représentative de la réalité du métier de développeur au sein d'une petite structure : le dialogue continu avec le client fait partie intégrante du travail, et savoir faire évoluer une architecture existante sans la réécrire entièrement est une compétence en soi.

Bilan humain et professionnel

Sur le plan humain, ce stage m'a permis de découvrir le fonctionnement quotidien d'une petite entreprise artisanale, très différente d'une structure informatique classique, et d'échanger régulièrement avec mon maître de stage sur ses attentes, ses contraintes et ses usages numériques. Cette expérience m'a également appris à expliquer des choix techniques (par exemple, pourquoi un dépôt de code ne peut pas être rendu public, ou pourquoi une fonctionnalité supplémentaire peut avoir un coût d'hébergement) à une personne non technicienne, de manière claire et sans jargon inutile - une compétence que je considère comme essentielle pour un futur professionnel de l'informatique, appelé à travailler aussi bien avec des collègues techniques que des utilisateurs finaux.

L'organisation hybride de ce stage, alternant présence sur site et travail à distance, m'a par ailleurs habitué à organiser mon temps de travail de manière autonome, tout en restant joignable et en rendant compte régulièrement de mon avancement à mon maître de stage.

Ouverture

À l'issue de ce stage, le Fournil des Corbières dispose d'un site vitrine fonctionnel, d'un espace d'administration autonome et d'un projet documenté, versé sur un dépôt Git, pouvant servir de point de départ à de futures évolutions (voir partie 4.5). Pour ma part, ce projet constitue une première réalisation concrète que je pourrai présenter, par exemple, lors de futurs entretiens de stage ou d'apprentissage, et qui m'a conforté dans mon souhait de poursuivre ma formation en BTS SIO, option SLAM, vers le développement d'applications.

Annexes

Annexe 1 - Arborescence finale du projet

L'arborescence ci-dessous présente l'organisation finale des fichiers et dossiers du projet, telle que décrite en partie 2.4 et complétée au fil du développement présenté en partie 3 et 4.

```
Le_Fournil_Projet/
   |-- acceuil.php                  (page d'accueil - point d'entree du site)
   |-- script.js                    (lightbox : agrandissement photo/video)
   |-- Dockerfile                    (configuration pour le deploiement Render)
   |-- .gitignore
   |
   |-- acceuil.css               (style general du site public)
   |
   |-- sql/
   |   |   `-- creation.sql              (script SQL de creation des tables)
   |
   |-- data/
   |   |   |-- boulangerie.db            (base de donnees SQLite : admin, produits)
   |   |   `-- .gitkeep
   |
   `-- fonction/
         |-- db.php                    (connexion PDO a la base de donnees)
         |-- login.php                 (formulaire de connexion admin)
         |-- auth.php                  (verification identifiant / mot de passe)
         |-- logout.php                (deconnexion de la session admin)
         |-- admin_panel.php           (tableau de bord de l'espace admin)
         |-- ajouter_produit.php       (ajout d'un produit + media)
         |-- modifier_produit.php      (modification d'un produit + media)
         |-- supprimer_produit.php     (suppression d'un produit + media)
         |-- voir_produits.php         (reorganisation des produits - drag & drop)
         |-- save_order.php            (enregistrement de l'ordre via AJAX)
         |-- creer_admin.php           (script de creation du compte admin)
         |-- maj_db.php                (script de migration : ajout de colonnes)
         |  `-- admin.css          (style de l'espace d'administration)
         |-- img/                      (photos des produits)
         |   |   `-- .gitkeep
         `-- video/                    (videos des produits)
               `-- .gitkeep
```

Annexe 2 - Schéma de la base de données

Le schéma ci-dessous illustre la structure des deux tables de la base de données boulangerie.db, telle que présentée en partie 2.5. Les lignes en gris indiquent, pour les colonnes concernées, les valeurs possibles utilisées dans le projet.

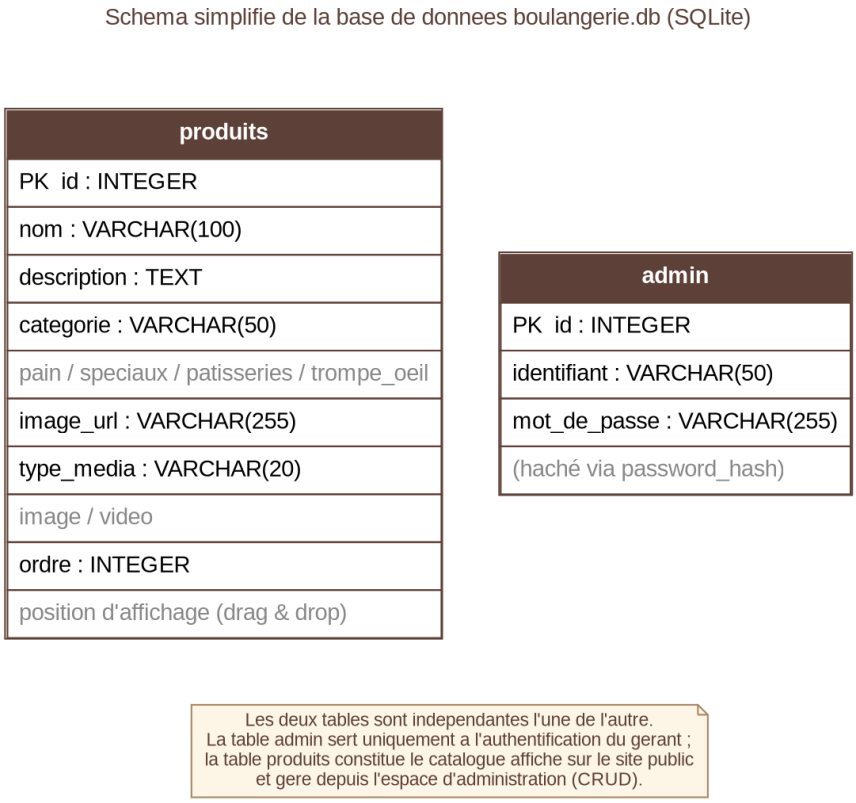


Figure 1 - Schéma simplifié de la base de données boulangerie.db.

Annexe 3 - Extrait du fichier de connexion à la base de données (db.php)

Le fichier ci-dessous correspond à la version finale de db.php (partie 2.5, 2.6 et 4.3) : il établit la connexion PDO vers le fichier SQLite situé sur le disque persistant (/data/boulangerie.db) et recrée automatiquement la structure des tables si elles n'existent pas encore, ce qui rend le script idempotent et autonome au premier démarrage.

```
<?php
// Chemin vers le disque persistant configure sur l'hebergeur
$db_path = '/data/boulangerie.db';

try {
    $db = new PDO('sqlite:' . $db_path);
    $db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    // Creation des tables si elles n'existent pas encore
    $db->exec("CREATE TABLE IF NOT EXISTS admin (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        identifiant VARCHAR(50) NOT NULL,
        mot_de_passe VARCHAR(255) NOT NULL
    )");

    $db->exec("CREATE TABLE IF NOT EXISTS produits (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        nom VARCHAR(100) NOT NULL,
        description TEXT,
        categorie VARCHAR(50),
        image_url VARCHAR(255),
        type_media VARCHAR(20) DEFAULT 'image',
        ordre INTEGER DEFAULT 0
    )");

} catch (PDOException $e) {
    die('Erreur de connexion : ' . $e->getMessage());
}
?>
```

Extrait - Fichier fonction/db.php (version finale).

Annexe 4 - Extrait du Dockerfile de déploiement

Le fichier ci-dessous correspond au Dockerfile utilisé pour le déploiement du projet sur la plateforme Render (partie 4.2 et 4.3) : il décrit l'image PHP/Apache utilisée, l'installation des extensions SQLite nécessaires, la configuration des droits d'écriture sur le dossier /data, ainsi que la copie du code source du projet.

```
# Image PHP officielle avec serveur Apache
FROM php:8.1-apache

# Installation des extensions necessaires pour SQLite
RUN apt-get update && apt-get install -y libsqlite3-dev \
    && docker-php-ext-install pdo pdo_sqlite

# Creation du dossier pour le disque persistant
RUN mkdir -p /data && chown -R www-data:www-data /data && chmod -R 777 /data

# Copie du code source du projet
COPY . /var/www/html/
RUN chown -R www-data:www-data /var/www/html

# Exposition du port HTTP
EXPOSE 80

# Demarrage du serveur Apache
CMD ["apache2-foreground"]
```

Extrait - Dockerfile utilisé pour le déploiement sur Render.